Lec14

Array

An Array is a special variable that can store multiple values in a single variable.

Example

Instead of creating many variables:

```
let student1 = "A";
let student2 = "B";
let student3 = "C";
```

We can use:

```
let students = ["A", "B", "C"];
```

Why use Array?

Store multiple values together

Easy data management

Faster searching and sorting

Useful in loops

Commonly used in web applications

Creating Array

Using Array Literal

```
let fruits = ["Apple", "Banana", "Mango"];
```

Using Array Constructor

```
let fruits = new Array("Apple", "Banana", "Mango");
```

Accessing Elements

```
let fruits = ["Apple", "Banana", "Mango"];


console.log(fruits[0]); // Apple

console.log(fruits[1]); // Banana

console.log(fruits[2]); // Mango
```

Properties

Length

Push

Pop

Unshift

Shift

Includes

Indexof

Join

Slice

splice

Looping through Array

for loop

```
for(let i=0;i<fruits.length;i++){
   console.log(fruits[i]);
}
```

for...of

```
for(let fruit of fruits){
   console.log(fruit);
}
```

forEach()

```
fruits.forEach(function(fruit){
   console.log(fruit);
});
```

Advanced Array Method

map()

filter()

reduce()

find()

some()

every()

String

A String is a sequence of characters.

Example

Let name=”Naman”;

String Immutability

Strings cannot be modified directly.

```
let str = "Hello";

str[0] = "Y"; // Won't change
```

Properties

Length

Touppercase

Tolowercase

Trim

Includes

Startswith

Endswith

indexof

Lastindexof

Slice

Substring

Replace

Replaceall

Split

Objects

An object stores data in key-value pairs.

Example

```
let student = {
   name: "A",
   age: 20,
   city: "Delhi"
};
```

Why Objects?

Real-world entities have multiple properties.

Example:

Student

- Name
- Age
- Roll Number
- Course

Accessing Properties

Dot Notation

student.name

Bracket Notation

student["name"]

Adding Properties

student.grade = "A";

Updating Properties

student.age = 21;

Deleting Properties

delete student.city;

Object Methods

```
let person = {
    name: "A",


    greet(){
        console.log("Hello");
    }
};
```

Looping through Objects

for...in

```
for(let key in student){
   console.log(key, student[key]);
}
```

Nested Objects

```
let student = {
   name: "John",

   address:{
      city:"Delhi",
      state:"Delhi"
   }
};
```

Array of Objects

```
let students = [
   {name:"John"},
   {name:"Emma"},
   {name:"Alex"}
];
```

Most common structure in projects.

ES6 BASICS

ES6 (ECMAScript 2015) introduced modern JavaScript features.

Let

Const

Arrow Functions

Template Literals

`Hello ${name}`

Default Parameters

```
function greet(name="Guest"){
   return name;
}
```

Destructuring

Array Destructuring

```
let [a,b] = [10,20];
```

Object Destructuring

```
let {name, age} = student;
```

Spread Operator (...)

Copies data.

```
let arr1 = [1,2];
let arr2 = [...arr1];
```

Rest Operator (...)

Collects values.

```
function sum(...nums){
}
```

Enhanced Object Literals

```
let name = "A";

let student = {
   name
};
```

Modules

Export:

export default Student;

Import:

import Student from "./Student";

Object Manipulation

Object Manipulation means creating, modifying, merging, copying, and transforming objects.

Merging Objects

```
let obj1 = {a:1};
let obj2 = {b:2};

let merged = {...obj1,...obj2};
```

Object.keys()

Returns all keys.

Object.keys(student);

Result:

["name","age"]

Object.values()

Returns values.

Object.values(student);

Object.entries()

Returns key-value pairs.

Object.entries(student);

Object.assign()

Copies objects.

let copy = Object.assign({}, student);

Spread Operator with Objects

let copy = {...student};

Cecking Property

in Operator

"name" in student

hasOwnProperty()

student.hasOwnProperty("name");

Object Freeze

Prevents modification.

Object.freeze(student);

Object Seal

Allows update but no add/delete.

Object.seal(student);

Deep vs Shallow Copy

Shallow Copy

let copy = {...student};

Nested objects still share reference.

Deep Copy

structuredClone(student);

Creates completely independent copy.

Task

Perform array, string and object-based data operations